# Assignment-4

* Problem Statement
  Write a C program to store first year percentage of students in an array. write a function for sorting array of floating point numbers in ascending order using bucket sort and display top five scores.

* objectives

1. Understand Bucket Sort.
2. Implement Bucket Sort using function.
3. Sort floating point percentages.
4. Display top fives scores.
5. Analyze complexity.

* Implementation
  - 64 bit open source linux or its derivatives.
  - Open Source C programming tool like gcc/ eclipse editer.

* Pseudo Code
  Accept, input, create buckets, sort buckets, ~~merge~~, display top five.

Start
1. Read number of students (N).

(2) Input the first year percentage of all students into array A.

(3) Create N empty buckets.
For each percentage in array A:
A. Find the appropriate bucket index.
B. Insert the percentage into the corresponding bucket.

(4) Sort elements of each bucket in ascending order.

(5) Merge all sorted buckets into the original array.

(6) Display the sorted percentages.

(7) Display the top five scores by printing last five elements of sorted array in descending order.

(8) Stop
end.

* Time Complexity

| operation | Time complexity |
|---|---|
| Accept input | O(n) |
| Create buckets | O(n) |
| Distribute element into bucket | O(n) |
| Sort individual bucket | O(n) |
| Display sorted array | O(n) |
| Display top five scores | O(1) |

* Conclusion:
Bucket sort efficiently sorts uniformly distributed floating - point percentages and helps identify the top five performers.

* FAQ's

1. What is Bucket Sort?
=> It is sorting algorithm that works by dividing elements into number of buckets, sorting the elements inside each bucket individually and then combining all the buckets to produce all the final sorted array. It is suitable for floating point numbers that are uniformly distributed over range.

2. Why use it for floating point data?
=>
Bucket Sort is well suited for floating point ~~numbers~~ because these values ~~usually~~ lie within a known range. This makes it easy to divide the range into buckets.
(1) Easy to distribute
(2) uniform distribution
(3) Efficient performance
(4) Suitable for decimal values.

3. What is bucket?

→ A bucket is a temporary container used to store elements that fall within particular range of values during bucket sort.

(1) Divides that data into several buckets.
(2) Sort each bucket separately
(3) Merges all buckets to get the final sorted array.

4. ~~Worts~~ worst case Complexity.

→ $O(n^2)$

The worst case occurs when all elements are placed into same bucket. Causing the internal sorting algorithm to take quadratic time.

5. Applications.

→

(1) Sorting Student percentages
(2) Sorting uniformly distributed data.
(3) Statistical and data analysis application.
(4) Image processing
(5) Scientific & engineering application.
(6) Database System.

10/10/2018

## C Program:

```
#include <stdio.h>

void bucketSort(float a[], int n)
{
    float bucket[100][100];
    int count[100] = {0};
    int i, j, k;
    float temp;

    for (i = 0; i < n; i++)
    {
        k = (int)a[i];
        bucket[k][count[k]] = a[i];
        count[k]++;
    }

    for (i = 0; i < 100; i++)
    {
        for (j = 0; j < count[i] - 1; j++)
        {
            int l;

            for (l = 0; l < count[i] - j - 1; l++)
            {
                if (bucket[i][l] > bucket[i][l + 1])
                {
                    temp = bucket[i][l];
                    bucket[i][l] = bucket[i][l + 1];
                    bucket[i][l + 1] = temp;
                }
            }
        }
    }

    k = 0;

    for (i = 0; i < 100; i++)
    {
        for (j = 0; j < count[i]; j++)
        {
            a[k] = bucket[i][j];
            k++;
        }
    }
}

void displayTopFive(float a[], int n)
{
    int i, limit;

    if (n < 5)
        limit = n;
    else
        limit = 5;

    printf("\nTop %d Scores:\n", limit);

    for (i = n - 1; i >= n - limit; i--)
    {
        printf("%.2f\n", a[i]);
    }
}

int main()
{
    float percentage[100];
    int n, i;

    printf("Enter number of students: ");
    scanf("%d", &n);

    printf("Enter percentages of students:\n");

    for (i = 0; i < n; i++)
    {
        scanf("%f", &percentage[i]);
```

```
    }

    bucketSort(percentage, n);

    printf("\nPercentages in Ascending Order:\n");

    for (i = 0; i < n; i++)
    {
        printf("%.2f ", percentage[i]);
    }

    displayTopFive(percentage, n);

    return 0;
}
```

## Output:

```
Enter number of students: 10
Enter percentages of students:
91 92 91 83 82 84 95 82 88 73

Percentages in Ascending Order:
73.00 82.00 82.00 83.00 84.00 88.00 91.00 91.00 92.00 95.00

Top 5 Scores:
95.00
92.00
91.00
91.00
88.00
```